

Pack 22 (Desktop) — Session Summary + Verifiable Consent Receipt at Disconnect (real code)

RemoteDesk · Pack 22 (Desktop) — Session Summary + Verifiable Consent Receipt at Disconnect (real code)

Codex / Claude Code handoff. Direct continuation of Desktop Packs 15 → 21. The permission lifecycle is complete; this slice produces the closing artifact: when a session ends, build a content-free **session summary** + a **consent receipt** the user can keep as proof of exactly what was shared, for how long, under what consent, and what control happened. The receipt is verifiable offline via a hash chain over the session's own audit timeline (Pack 17/18). Build order: shared receipt contract → summary builder → receipt builder + offline verifier → disconnect wiring → UI → tests.

Scope: a tamper-evident, non-sensitive record of the session, derived entirely from data already collected (timeline, permissions, grant). **No payloads, keystrokes, file contents, IPs, or secrets in the receipt or summary. Verifiable without the server. Host stays source of truth. No native executor.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]		
packages/shared/src/permissions/sessionSummary.ts		
packages/shared/src/permissions/consentReceipt.ts		
apps/desktop/src/renderer/src/services/sessionSummaryBuilder.ts		
apps/desktop/src/renderer/src/services/consentReceiptService.ts		
apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx		
apps/desktop/src/renderer/src/components/ConsentReceiptBadge.tsx		
tests/permissions/sessionSummary.test.ts		
tests/permissions/consentReceipt.test.ts		
tests/permissions/consentReceiptService.test.ts		
MODIFIED [MODIFY]		
packages/shared/src/permissions/index.ts		(export sessionSummary + consentReceipt)
apps/desktop/src/renderer/src/services/reconnectPolicyController.ts		(capture end-of-session on terminal failure)
apps/desktop/src/renderer/src/components/RemoteSessionView.tsx		(summary dialog on disconnect)
NOT TOUCHED (intentionally)		
apps/desktop/src/main/index.ts		(no native executor)
apps/desktop/src/preload/index.ts		(no new bridge)
apps/desktop/src/renderer/src/services/webrtc.ts		(Pack 15/17 gates unchanged)
apps/desktop/src/renderer/src/services/permissionState.ts		(Pack 15/16 unchanged)
apps/api/**	apps/web/**	(receipt is user-held; optional server copy out of scope)

Part A — Shared: summary + receipt contracts

FILE: packages/shared/src/permissions/sessionSummary.ts

```
import type { SessionAuditEvent } from './sessionAuditEvents';
import type { SessionPermissionSet } from './devicePolicy';

/**
 * A content-free summary of a finished session, derived from data already
 * collected during the session (audit timeline + final permission set). NO
 * payloads, keystrokes, file contents, IPs, or secrets — only counts, durations,
 * and which capabilities were ever active.
 */
export interface SessionSummary {
  deviceId: string;
  startedAt: string;
  endedAt: string;
  durationSeconds: number;
  /** Capabilities that were enabled at any point during the session. */
  capabilitiesUsed: {
    remoteInput: boolean;
    clipboardSync: boolean;
    fileTransfer: boolean;
  };
  /** Whether the session ran unattended (auto-accepted under a grant). */
  unattended: boolean;
  /** Aggregate counts from the audit timeline. */
  counts: {
    inputBlocked: number;
    inputRateLimited: number;
    emergencyStops: number;
    controlGrants: number;
    controlRevokes: number;
  };
  /** How the session ended. */
  endReason: 'user_disconnect' | 'expired' | 'reconnect_failed' | 'unknown';
}

/** Count events of a given type in the timeline. */
export function countEvents(events: SessionAuditEvent[], type: string): number {
  return events.filter((e) => e.type === type).length;
}

/**
 * Build a summary from the audit timeline + bounds. Pure: capability usage is
 * inferred from control/feature events + the peak permission set seen.
 */
export function buildSessionSummary(input: {
  deviceId: string;
  startedAt: string;
  endedAt: string;
  events: SessionAuditEvent[];
  peakPermissions: Pick<SessionPermissionSet, 'remoteInput' | 'clipboardSync' | 'fileTransfer'> | null;
  unattended: boolean;
  endReason: SessionSummary['endReason'];
}): SessionSummary {
  const durationSeconds = Math.max(
    0,
    Math.round((Date.parse(input.endedAt) - Date.parse(input.startedAt)) / 1000),
  );
}
```

```

);
return {
  deviceId: input.deviceId,
  startedAt: input.startedAt,
  endedAt: input.endedAt,
  durationSeconds,
  capabilitiesUsed: {
    remoteInput: (input.peakPermissions?.remoteInput ?? 'disabled') === 'enabled',
    clipboardSync: input.peakPermissions?.clipboardSync ?? false,
    fileTransfer: input.peakPermissions?.fileTransfer ?? false,
  },
  unattended: input.unattended,
  counts: {
    inputBlocked: countEvents(input.events, 'input.blocked'),
    inputRateLimited: countEvents(input.events, 'input.rate_limited'),
    emergencyStops: countEvents(input.events, 'emergency_stop.engaged'),
    controlGrants: countEvents(input.events, 'control.granted'),
    controlRevokes: countEvents(input.events, 'control.revoked'),
  },
  endReason: input.endReason,
};
}

```

FILE: packages/shared/src/permissions/consentReceipt.ts

```

import type { SessionSummary } from './sessionSummary';
import type { SessionAuditEvent } from './sessionAuditEvents';

/**
 * A verifiable consent receipt: the session summary + a hash CHAIN over the
 * session's audit timeline. Each line is hashed with the previous hash, so any
 * insertion/removal/reordering breaks the chain. The receipt is content-free and
 * verifiable OFFLINE (no server) by recomputing the chain from the line items.
 */
export interface ReceiptLineItem {
  /** Sequence in the session timeline. */
  seq: number;
  /** Non-sensitive event type + ISO time (no payloads). */
  type: string;
  at: string;
  /** Hash linking this line to the previous (hex). */
  hash: string;
}

export interface ConsentReceipt {
  receiptId: string;
  summary: SessionSummary;
  lineItems: ReceiptLineItem[];
  /** Final chain hash = the merkle-ish root of the whole timeline. */
  rootHash: string;
  issuedAt: string;
  /** Schema/version tag so verifiers can evolve. */
  version: 1;
}

/** Canonical per-line preimage. Order-stable, non-sensitive fields only. */

```

```

export function linePreimage(prevHash: string, seq: number, type: string, at: string): string {
    return `${prevHash}|${seq}|${type}|${at}`;
}

/**
 * Build the chained line items from the audit timeline using an injected hash fn
 * (so shared stays runtime-agnostic; desktop passes a SHA-256). Returns the items
 * + the root hash. Pure given the hash function.
 */
export async function buildChain(
    events: SessionAuditEvent[],
    sha256Hex: (input: string) => Promise<string>,
    genesis = 'remotedesk-consent-genesis',
): Promise<{ lineItems: ReceiptLineItem[]; rootHash: string }> {
    let prev = genesis;
    const lineItems: ReceiptLineItem[] = [];
    for (let i = 0; i < events.length; i++) {
        const e = events[i];
        const hash = await sha256Hex(linePreimage(prev, i, e.type, e.at));
        lineItems.push({ seq: i, type: e.type, at: e.at, hash });
        prev = hash;
    }
    return { lineItems, rootHash: prev };
}

/**
 * Offline verification: recompute the chain from the receipt's own line items and
 * confirm every link + the root. Returns the first broken seq, or null if intact.
 * Requires NO server and NO original events — just the receipt.
 */
export async function verifyReceipt(
    receipt: ConsentReceipt,
    sha256Hex: (input: string) => Promise<string>,
    genesis = 'remotedesk-consent-genesis',
): Promise<{ valid: boolean; firstBrokenSeq: number | null }> {
    let prev = genesis;
    for (const item of receipt.lineItems) {
        const expected = await sha256Hex(linePreimage(prev, item.seq, item.type, item.at));
        if (expected !== item.hash) return { valid: false, firstBrokenSeq: item.seq };
        prev = expected;
    }
    if (prev !== receipt.rootHash) return { valid: false, firstBrokenSeq: null };
    return { valid: true, firstBrokenSeq: null };
}

```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```

export * from './devicePolicy';
export * from './policyEvaluator';
export * from './sessionApproval';
export * from './inputAction';
export * from './sessionAuditEvents';
export * from './reconnect';
export * from './fileTransferState';
export * from './clipboardSync';
export * from './deviceFingerprint';

```

```
export * from './trustDecision';
export * from './unattendedGrant';
export * from './autoAccept';
export * from './sessionSummary'; // ADD
export * from './consentReceipt'; // ADD
```

Part B — Desktop: session summary builder

FILE: apps/desktop/src/renderer/src/services/sessionSummaryBuilder.ts

```
import {
  buildSessionSummary,
  type SessionAuditEvent,
  type SessionPermissionSet,
  type SessionSummary,
} from '@remotedesk/shared';
import type { SessionTimelineStore } from './sessionTimelineStore';

/**
 * Builds a SessionSummary at disconnect from the Pack 18 timeline store + the
 * peak permission set observed during the session. Tracks the peak (most-enabled)
 * permissions so the summary reflects what was EVER active, not just the final state.
 */
export class SessionSummaryBuilder {
  private startedAt = new Date().toISOString();
  private peak: Pick<SessionPermissionSet, 'remoteInput' | 'clipboardSync' | 'fileTransfer'> | null = null;
  private unattended = false;

  constructor(private readonly timeline: SessionTimelineStore, private readonly deviceId: string) {}

  markStarted(at: string = new Date().toISOString()): void {
    this.startedAt = at;
  }

  markUnattended(unattended: boolean): void {
    this.unattended = this.unattended || unattended;
  }

  /** Feed every permission snapshot; we keep the most-permissive seen. */
  observePermissions(p: SessionPermissionSet): void {
    const cur = this.peak;
    this.peak = {
      remoteInput: p.remoteInput === 'enabled' || cur?.remoteInput === 'enabled' ? 'enabled' : 'disabled',
      clipboardSync: !!p.clipboardSync || !!cur?.clipboardSync,
      fileTransfer: !!p.fileTransfer || !!cur?.fileTransfer,
    };
  }

  build(endReason: SessionSummary['endReason'], endedAt: string = new Date().toISOString()): SessionSummary {
    const events: SessionAuditEvent[] = [...this.timeline.all()];
    return buildSessionSummary({
      deviceId: this.deviceId,
      startedAt: this.startedAt,
      endedAt,
      events,
    });
  }
}
```

```

        peakPermissions: this.peak,
        unattended: this.unattended,
        endReason,
    });
}
}

```

Part C — Desktop: consent receipt service

FILE: apps/desktop/src/renderer/src/services/consentReceiptService.ts

```

import {
    buildChain,
    verifyReceipt,
    type ConsentReceipt,
    type SessionAuditEvent,
    type SessionSummary,
} from '@remotedesk/shared';

/**
 * Issues + verifies consent receipts on the desktop using the renderer's SHA-256.
 * The receipt is user-held (downloadable JSON); a server copy is out of scope.
 * Verification works entirely offline from the receipt's own line items.
 */
export class ConsentReceiptService {
    /** Build a receipt from a finished session's summary + audit timeline. */
    async issue(summary: SessionSummary, events: SessionAuditEvent[]): Promise<ConsentReceipt> {
        const { lineItems, rootHash } = await buildChain(events, sha256Hex);
        return {
            receiptId: cryptoRandomId(),
            summary,
            lineItems,
            rootHash,
            issuedAt: new Date().toISOString(),
            version: 1,
        };
    }

    /** Offline-verify a receipt (recompute the chain). */
    verify(receipt: ConsentReceipt): Promise<{ valid: boolean; firstBrokenSeq: number | null }> {
        return verifyReceipt(receipt, sha256Hex);
    }

    /** Trigger a browser download of the receipt JSON. */
    download(receipt: ConsentReceipt): void {
        const blob = new Blob([JSON.stringify(receipt, null, 2)], { type: 'application/json' });
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = `consent-receipt-${receipt.receiptId}.json`;
        a.click();
        URL.revokeObjectURL(url);
    }
}

```

```

async function sha256Hex(input: string): Promise<string> {
  const data = new TextEncoder().encode(input);
  const digest = await crypto.subtle.digest('SHA-256', data);
  return Array.from(new Uint8Array(digest), (b) => b.toString(16).padStart(2, '0')).join('');
}

function cryptoRandomId(): string {
  const bytes = new Uint8Array(16);
  crypto.getRandomValues(bytes);
  return Array.from(bytes, (b) => b.toString(16).padStart(2, '0')).join('');
}

```

Part D — Desktop: disconnect wiring

FILE: apps/desktop/src/renderer/src/services/reconnectPolicyController.ts [MODIFY]

```

// ...existing Pack 18/19 imports...

// ---- ADD: optional end-of-session callback so a terminal 'failed' phase can
// trigger the summary + receipt flow. Wire in RemoteSessionView. ----
//
// export interface ReconnectDeps {
//   ...existing...
//   onSessionEnded?: (reason: 'reconnect_failed') => void; // ADD
// }
//
// In fail(reason) and in onTransportRestored() when it transitions to 'failed':
//   this.deps.onSessionEnded?.('reconnect_failed');
//
// This is the ONLY change here: a permanently-failed reconnect is treated as a
// session end so the user still gets a summary + receipt.

```

Part E — UI

FILE: apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx

```

import React, { useState } from 'react';
import type { ConsentReceipt, SessionSummary } from '@remotedesk/shared';
import type { ConsentReceiptService } from '../services/consentReceiptService';

const REASON_LABEL: Record<string, string> = {
  user_disconnect: 'You ended the session',
  expired: 'Session time limit reached',
  reconnect_failed: 'Connection lost and could not safely resume',
  unknown: 'Session ended',
};

/**
 * Shown at disconnect. Displays the content-free summary and lets the user
 * download + verify a consent receipt. Screen view is already gone by this point.
 */
export function SessionSummaryDialog({
  summary,
  receipt,

```

```

    service,
    onClose,
  }: {
    summary: SessionSummary;
    receipt: ConsentReceipt | null;
    service: ConsentReceiptService;
    onClose: () => void;
  }) {
    const [verifyState, setVerifyState] = useState<'idle' | 'valid' | 'invalid'>('idle');

    const mins = Math.floor(summary.durationSeconds / 60);
    const secs = summary.durationSeconds % 60;

    const verify = async () => {
      if (!receipt) return;
      const r = await service.verify(receipt);
      setVerifyState(r.valid ? 'valid' : 'invalid');
    };

    return (
      <div className="session-summary-dialog" role="dialog" aria-modal="true">
        <h2>Session ended</h2>
        <p className="session-summary-dialog__reason">{REASON_LABEL[summary.endReason]}</p>

        <dl className="session-summary-dialog__grid">
          <dt>Duration</dt><dd>{mins}m {secs}s</dd>
          <dt>Mode</dt><dd>{summary.unattended ? 'Unattended (pre-authorized)' : 'Attended'}</dd>
          <dt>Remote input</dt><dd>{summary.capabilitiesUsed.remoteInput ? 'used' : 'not used'}</dd>
          <dt>Clipboard</dt><dd>{summary.capabilitiesUsed.clipboardSync ? 'used' : 'not used'}</dd>
          <dt>File transfer</dt><dd>{summary.capabilitiesUsed.fileTransfer ? 'used' : 'not used'}</dd>
          <dt>Input blocked</dt><dd>{summary.counts.inputBlocked}</dd>
          <dt>Emergency stops</dt><dd>{summary.counts.emergencyStops}</dd>
        </dl>

        <div className="session-summary-dialog__actions">
          {receipt && (
            <>
              <button type="button" className="btn btn--primary" onClick={() => service.download(receipt)}>
                Download consent receipt
              </button>
              <button type="button" className="btn btn--ghost" onClick={verify}>
                {verifyState === 'idle' ? 'Verify receipt' : verifyState === 'valid' ? '✓ Verified intact' : '⊠
Verification failed'}
              </button>
            </>
          )}
          <button type="button" className="btn" onClick={onClose}>Close</button>
        </div>
      </div>
    );
  }
}

```

FILE: apps/desktop/src/renderer/src/components/ConsentReceiptBadge.tsx

```

import React from 'react';
import type { ConsentReceipt } from '@remotedesk/shared';

```



```

/**
 * Tiny badge showing a receipt was issued + its short root hash. Lets the user
 * recognize the receipt without opening it. Non-sensitive (root hash only).
 */
export function ConsentReceiptBadge({ receipt }: { receipt: ConsentReceipt | null }) {
  if (!receipt) return null;
  const short = receipt.rootHash.slice(0, 12);
  return (
    <span className="consent-receipt-badge" title={`Receipt ${receipt.receiptId}`}>
      Receipt issued · {short}...
    </span>
  );
}

```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```

// ...existing imports + Pack 15→21 additions...
import { SessionSummaryDialog } from '../SessionSummaryDialog';
import { SessionSummaryBuilder } from '../services/sessionSummaryBuilder';
import { ConsentReceiptService } from '../services/consentReceiptService';
import type { ConsentReceipt, SessionSummary } from '@remotedesk/shared';

// ---- ADD: construct once at session start ----
// const summaryBuilder = useMemo(() => new SessionSummaryBuilder(sessionTimelineStore, deviceId), []);
// const receiptService = useMemo(() => new ConsentReceiptService(), []);
// const [endSummary, setEndSummary] = useState<SessionSummary | null>(null);
// const [endReceipt, setEndReceipt] = useState<ConsentReceipt | null>(null);
//
// summaryBuilder.markStarted();
// // feed it the same data the gates see:
// //   on every permission snapshot: summaryBuilder.observePermissions(permissions)
// //   when auto-accepted (Pack 21): summaryBuilder.markUnattended(true)

// ---- ADD: a single end-of-session handler used by all end paths ----
// async function endSession(reason: SessionSummary['endReason']) {
//   const summary = summaryBuilder.build(reason);
//   const receipt = await receiptService.issue(summary, [...sessionTimelineStore.all()]);
//   setEndSummary(summary);
//   setEndReceipt(receipt);
// }
//
// Wire end paths:
// - user clicks Disconnect      -> endSession('user_disconnect')
// - SessionExpiryWatcher fires  -> endSession('expired')
// - reconnectController onSessionEnded('reconnect_failed') -> endSession('reconnect_failed')

// ---- ADD to render: the summary dialog once the session has ended ----
// {endSummary && (
//   <SessionSummaryDialog
//     summary={endSummary}
//     receipt={endReceipt}
//     service={receiptService}
//     onClose={() => { setEndSummary(null); /* then actually tear down the view */ }}
//   />
// )}

```

```
//  
// All Pack 15→21 UI stays as-is. The receipt is built from the SAME timeline the  
// gates + audit forwarder already use, so it can't disagree with what happened.
```

Part F — Tests

FILE: tests/permissions/sessionSummary.test.ts

```
import { describe, it, expect } from 'vitest';  
import { buildSessionSummary, countEvents, makeAuditEvent } from '@remotedesk/shared';  
  
const events = [  
  makeAuditEvent('control.granted', {}, new Date('2026-06-15T10:00:00Z')),  
  makeAuditEvent('input.blocked', { seq: 1 }, new Date('2026-06-15T10:01:00Z')),  
  makeAuditEvent('input.blocked', { seq: 2 }, new Date('2026-06-15T10:01:01Z')),  
  makeAuditEvent('emergency_stop.engaged', {}, new Date('2026-06-15T10:02:00Z')),  
];  
  
describe('sessionSummary', () => {  
  it('counts events by type', () => {  
    expect(countEvents(events, 'input.blocked')).toBe(2);  
    expect(countEvents(events, 'emergency_stop.engaged')).toBe(1);  
  });  
  
  it('builds a summary with duration + capabilities + counts', () => {  
    const s = buildSessionSummary({  
      deviceId: 'dev-1',  
      startedAt: '2026-06-15T10:00:00.000Z',  
      endedAt: '2026-06-15T10:05:00.000Z',  
      events,  
      peakPermissions: { remoteInput: 'enabled', clipboardSync: true, fileTransfer: false },  
      unattended: true,  
      endReason: 'user_disconnect',  
    });  
    expect(s.durationSeconds).toBe(300);  
    expect(s.capabilitiesUsed).toEqual({ remoteInput: true, clipboardSync: true, fileTransfer: false });  
    expect(s.unattended).toBe(true);  
    expect(s.counts.inputBlocked).toBe(2);  
    expect(s.counts.emergencyStops).toBe(1);  
    expect(s.endReason).toBe('user_disconnect');  
  });  
  
  it('treats null peak permissions as nothing used (fail closed)', () => {  
    const s = buildSessionSummary({  
      deviceId: 'dev-1', startedAt: '2026-06-15T10:00:00.000Z', endedAt: '2026-06-15T10:00:30.000Z',  
      events: [], peakPermissions: null, unattended: false, endReason: 'unknown',  
    });  
    expect(s.capabilitiesUsed).toEqual({ remoteInput: false, clipboardSync: false, fileTransfer: false });  
    expect(s.durationSeconds).toBe(30);  
  });  
});
```

FILE: tests/permissions/consentReceipt.test.ts

```

import { describe, it, expect } from 'vitest';
import { buildChain, verifyReceipt, linePreimage, makeAuditEvent, type ConsentReceipt, type SessionSummary } from
 '@remotedesk/shared';
import { createHash } from 'node:crypto';

// Node sha256 for the test (desktop uses Web Crypto; same algorithm).
const sha256Hex = async (input: string) => createHash('sha256').update(input).digest('hex');

const events = [
  makeAuditEvent('control.granted', {}, new Date('2026-06-15T10:00:00Z')),
  makeAuditEvent('input.blocked', {}, new Date('2026-06-15T10:01:00Z')),
  makeAuditEvent('emergency_stop.engaged', {}, new Date('2026-06-15T10:02:00Z')),
];

const summary: SessionSummary = {
  deviceId: 'dev-1', startedAt: '2026-06-15T10:00:00.000Z', endedAt: '2026-06-15T10:05:00.000Z',
  durationSeconds: 300, capabilitiesUsed: { remoteInput: true, clipboardSync: false, fileTransfer: false },
  unattended: false, counts: { inputBlocked: 1, inputRateLimited: 0, emergencyStops: 1, controlGrants: 1, controlRevokes:
0 },
  endReason: 'user_disconnect',
};

async function makeReceipt(): Promise<ConsentReceipt> {
  const { lineItems, rootHash } = await buildChain(events, sha256Hex);
  return { receiptId: 'r1', summary, lineItems, rootHash, issuedAt: '2026-06-15T10:05:01.000Z', version: 1 };
}

describe('consentReceipt', () => {
  it('builds a chain where each line links to the previous', async () => {
    const { lineItems } = await buildChain(events, sha256Hex);
    expect(lineItems).toHaveLength(3);
    expect(lineItems[0].seq).toBe(0);
    // line 1 hash must equal sha256(line0.hash | 1 | type | at)
    const expected = await sha256Hex(linePreimage(lineItems[0].hash, 1, lineItems[1].type, lineItems[1].at));
    expect(lineItems[1].hash).toBe(expected);
  });

  it('verifies an intact receipt offline', async () => {
    const receipt = await makeReceipt();
    expect(await verifyReceipt(receipt, sha256Hex)).toEqual({ valid: true, firstBrokenSeq: null });
  });

  it('detects a tampered line (reorder/edit)', async () => {
    const receipt = await makeReceipt();
    receipt.lineItems[1] = { ...receipt.lineItems[1], type: 'control.revoked' }; // tamper
    const r = await verifyReceipt(receipt, sha256Hex);
    expect(r.valid).toBe(false);
    expect(r.firstBrokenSeq).toBe(1);
  });

  it('detects a tampered root hash', async () => {
    const receipt = await makeReceipt();
    receipt.rootHash = 'deadbeef';
    const r = await verifyReceipt(receipt, sha256Hex);
    expect(r.valid).toBe(false);
    expect(r.firstBrokenSeq).toBeNull();
  });
});

```

```

});

it('verifies an empty timeline (root == genesis)', async () => {
  const { lineItems, rootHash } = await buildChain([], sha256Hex);
  expect(lineItems).toHaveLength(0);
  const receipt: ConsentReceipt = { receiptId: 'r0', summary, lineItems, rootHash, issuedAt: '', version: 1 };
  expect((await verifyReceipt(receipt, sha256Hex)).valid).toBe(true);
});
});

```

FILE: tests/permissions/consentReceiptService.test.ts

```

import { describe, it, expect, beforeAll } from 'vitest';
import { webcrypto } from 'node:crypto';
import { ConsentReceiptService } from '../../apps/desktop/src/renderer/src/services/consentReceiptService';
import { makeAuditEvent, buildSessionSummary, type SessionSummary } from '@remotedesk/shared';

// Provide Web Crypto in the Node test env (desktop relies on crypto.subtle).
beforeAll(() => {
  if (!(globalThis as any).crypto) (globalThis as any).crypto = webcrypto as any;
});

function summary(): SessionSummary {
  return buildSessionSummary({
    deviceId: 'dev-1', startedAt: '2026-06-15T10:00:00.000Z', endedAt: '2026-06-15T10:03:00.000Z',
    events: [], peakPermissions: { remoteInput: 'enabled', clipboardSync: false, fileTransfer: false },
    unattended: false, endReason: 'user_disconnect',
  });
}

describe('ConsentReceiptService', () => {
  it('issues a receipt that verifies as intact', async () => {
    const svc = new ConsentReceiptService();
    const events = [makeAuditEvent('control.granted'), makeAuditEvent('control.revoked')];
    const receipt = await svc.issue(summary(), events);
    expect(receipt.lineItems).toHaveLength(2);
    expect((await svc.verify(receipt)).valid).toBe(true);
  });

  it('a tampered receipt fails verification', async () => {
    const svc = new ConsentReceiptService();
    const receipt = await svc.issue(summary(), [makeAuditEvent('control.granted')]);
    receipt.lineItems[0] = { ...receipt.lineItems[0], at: '1999-01-01T00:00:00.000Z' };
    expect((await svc.verify(receipt)).valid).toBe(false);
  });
});

```

Part G — Reports

FILE: [generated-pack22-desktop-consent-receipt-merge-summary.md](#)

Pack 22 (Desktop) — Session Summary + Consent Receipt • Merge Summary

What this slice does

Continues Desktop Packs 15 → 21. Produces the closing artifact of a session:

1. SessionSummary — content-free recap (duration, capabilities ever used, attended/unattended, audit counts, end reason), derived from the Pack 18 timeline.
2. ConsentReceipt — the summary + a SHA-256 hash CHAIN over the audit timeline, so any edit/reorder/removal breaks the chain. Verifiable OFFLINE from the receipt alone (no server, no original events).
3. UI — a session-summary dialog at disconnect with download + verify, and a receipt badge.

Files created (9)

- packages/shared/src/permissions/sessionSummary.ts
- packages/shared/src/permissions/consentReceipt.ts
- apps/desktop/src/renderer/src/services/sessionSummaryBuilder.ts
- apps/desktop/src/renderer/src/services/consentReceiptService.ts
- apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx
- apps/desktop/src/renderer/src/components/ConsentReceiptBadge.tsx
- tests/permissions/sessionSummary.test.ts
- tests/permissions/consentReceipt.test.ts
- tests/permissions/consentReceiptService.test.ts

Files modified (3)

- packages/shared/src/permissions/index.ts — export sessionSummary + consentReceipt.
- apps/desktop/src/renderer/src/services/reconnectPolicyController.ts — onSessionEnded on terminal failure.
- apps/desktop/src/renderer/src/components/RemoteSessionView.tsx — summary dialog + receipt issuance at all end paths.

Files intentionally NOT touched

- main/index.ts, preload/index.ts — no native executor / no bridge.
- webrtc.ts, permissionState.ts — Pack 15→17 gates unchanged.
- apps/api/**, apps/web/** — receipt is user-held; optional server copy is out of scope.

Remaining manual wiring risks

1. Construct SessionSummaryBuilder + ConsentReceiptService at session start; feed observePermissions on each snapshot and markUnattended on auto-accept.
2. Route ALL end paths (user disconnect / expiry / reconnect-failed) through one endSession(reason).
3. Show SessionSummaryDialog before tearing down the view.
4. Optional: also POST the receipt to the API for an org copy (out of scope here).

Verify

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: end a session -> summary dialog shows correct duration/capabilities/counts; download receipt, edit a line in the JSON, re-import + verify -> fails at the edited seq.

FILE: [generated-pack22-desktop-consent-receipt-code-review.md](#)

Pack 22 (Desktop) — Code Review Notes

Strengths

- Receipt is built from the SAME audit timeline the gates + forwarder use, so it cannot disagree with what actually happened.
- Hash chain makes tampering detectable offline: edit/reorder/remove any line and verification pinpoints the first broken seq.
- Content-free by construction: only event type + ISO time + counts + durations; no payloads/keys/files/IPs.
- Hash fn is injected into shared (runtime-agnostic); desktop supplies Web Crypto, tests supply Node crypto — same algorithm.
- Peak-permission tracking means the summary reflects what was EVER enabled, not just the final state.

Reviewer checklist

- [] observePermissions called on every snapshot; markUnattended on Pack 21 auto-accept.
- [] All end paths funnel through one endSession(reason).
- [] Summary dialog renders before view teardown so the user can download.
- [] Receipt JSON contains no sensitive fields (verified: type/at/hash + summary counts only).
- [] Web Crypto available in the renderer (crypto.subtle).

Safety

- No payloads, keystrokes, file names/contents, or IPs in summary or receipt.
- Verification needs no server and no original events — just the receipt.
- Native OS execution remains disabled/no-op (untouched).

FILE: generated-pack22-desktop-consent-receipt-manifest.json

```
{
  "pack": "pack22-desktop-consent-receipt",
  "title": "Session Summary + Verifiable Consent Receipt at Disconnect",
  "actualFileCount": 12,
  "filesCreated": [
    "packages/shared/src/permissions/sessionSummary.ts",
    "packages/shared/src/permissions/consentReceipt.ts",
    "apps/desktop/src/renderer/src/services/sessionSummaryBuilder.ts",
    "apps/desktop/src/renderer/src/services/consentReceiptService.ts",
    "apps/desktop/src/renderer/src/components/SessionSummaryDialog.tsx",
    "apps/desktop/src/renderer/src/components/ConsentReceiptBadge.tsx",
    "tests/permissions/sessionSummary.test.ts",
    "tests/permissions/consentReceipt.test.ts",
    "tests/permissions/consentReceiptService.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/desktop/src/renderer/src/services/reconnectPolicyController.ts",
    "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/src/main/index.ts",
    "apps/desktop/src/preload/index.ts",
    "apps/desktop/src/renderer/src/services/webrtc.ts",
    "apps/desktop/src/renderer/src/services/permissionState.ts",
    "apps/api/**",
    "apps/web/**"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "react", "vitest"],
    "assumedFromProject": [
      "Desktop Pack 18 SessionTimelineStore",
      "Desktop Pack 21 auto-accept (for unattended flag)",
      "web crypto (crypto.subtle) in the renderer",
      "existing Disconnect action + Pack 16 expiry watcher"
    ]
  },
  "knownLimitations": [
    "Receipt is user-held; an org-side server copy is out of scope (optional follow-up).",
    "Chain is per-session, in-memory until issued; not persisted mid-session.",
    "Receipt proves the timeline is internally consistent; cross-device signing/attestation is a later step.",
    "Native OS input execution remains disabled/no-op."
  ]
}
```

```
],  
  "remainingManualWiringRisks": [  
    "Feed observePermissions + markUnattended during the session.",  
    "Funnel all end paths through one endSession(reason).",  
    "Render the summary dialog before tearing down the session view."  
  ],  
  "estimatedCompletionAfterCarefulMerge": "74-80%"  
}
```

Desktop Pack 22 done: 12 files (9 created, 3 modified). The closing artifact of a session: a content-free summary (duration, which capabilities were ever active, attended vs unattended, audit counts, how it ended) plus a verifiable consent receipt, a SHA-256 hash chain over the session's own audit timeline that the user can download and verify offline.

The clean bit: the receipt is built from the exact same timeline the gates and audit forwarder already use, so it physically can't disagree with what happened, and the hash chain means editing, reordering, or removing any line breaks verification and pinpoints the first broken entry, with no server and no original events needed. It's content-free by construction: event type + ISO time + counts only, never payloads, keystrokes, file names, or IPs.

That's the desktop permission lifecycle fully closed, trust established (20), pre-authorized (21), enforced (15), consented (16), funneled (17), reconnect-safe (18/19), and now receipted (22). Next natural slice = the web-side admin UI to create/manage unattended grants + trust + view receipts, which moves back into apps/web. Bolo.